

MiniMind 预训练里的梯度累积与SkipBatchSampler

本次补充：尾部不足累积窗口时为什么仍除以 `K`、checkpoint 的真实恢复边界、“墙钟时间”和四类非完全等价来源，以及“中断在一个 micro-batch 中途”时 `SkipBatchSampler` 到底会不会跳过数据。

本文只解释 MiniMind 个人复现仓库中的预训练链路，重点回答两个问题：

- 1. 为什么预训练默认设置了 `accumulation_steps=8`。
- 2. `SkipBatchSampler` 到底跳过了什么，它在恢复训练里起什么作用。

事实边界先说清楚：

- 当前本地代码事实：本仓库 `trainer/train_pretrain.py`、`trainer/trainer_utils.py`、`dataset/lm_dataset.py` 与本机上游引用仓库对应文件逐字一致，本轮用 `cmp` 检查结果为 0。
- 上游引用事实：本机上游引用仓库位于 `../../../references/minimind`，当前提交为 `512eed0b6556e741d80864f054d45d271459772a`。
- 本机已验证事实：已有极小 CPU 单 batch 实验验证过 `JSONL -> tokenizer -> Dataset -> DataLoader -> forward -> loss -> backward -> optimizer.step -> checkpoint/resume` 最短闭环，见 [实验记录](#)。该实验没有验证正式 `train_pretrain.py` 的 AMP、DDP、梯度累积和长时间 GPU 训练。
- 工程判断：梯度累积主要服务显存受限场景下的有效 batch 放大；`SkipBatchSampler` 主要服务断点续训时的数据进度对齐。它们都不是提升模型智商的魔法按钮，而是训练工程里的秩序维护工具。

先把训练循环里的几个动词说成人话

如果不熟训练代码，先不要急着看 `accumulation_steps`。MiniMind 预训练每一小批数据进来时，核心动作可以翻译成下面这张表：

代码动作	人话解释	发生在 MiniMind 哪里
<code>forward</code>	把一批 <code>input_ids</code> 喂给模型，让模型根据当前参数猜“下一个 token 是谁”，输出每个位置对整个词表的打分 <code>logits</code> 。	train_pretrain.py 、 model_minimind.py
算 <code>loss</code>	把模型的猜测和正确答案 <code>labels</code> 对比，得到一个“错得有多离谱”的数字。数字越小，说明当前这批数据上猜得越接近。	model_minimind.py
<code>loss.backward()</code>	根据这个错误数字，反推出每个参数应该往哪个方向改。它只是把“修改建议”写到每个参数的 <code>.grad</code> 里，还没有真正改参数。	train_pretrain.py
<code>optimizer.step()</code>	优化器读取 <code>.grad</code> 里的修改建议，真正改变模型参数。	train_pretrain.py
<code>optimizer.zero_grad()</code>	清空刚才用过的修改建议，避免下一轮训练把旧建议和新建议混在一起。	train_pretrain.py

可以把一次普通训练想成“做题、批改、改答案、擦掉草稿”。`forward` 是做题，`loss` 是批改分数，`backward` 是写出哪里该改，`optimizer.step()` 是真的改答案，`optimizer.zero_grad()` 是把这次批改意见从草稿纸上擦掉。梯度累积的特殊点在于：MiniMind 不是每次批改后都立刻改答案，而是先攒 8 次批改意见，再统一改一次。

显存也先用人话理解：显存不是“模型聪明程度”，而是 GPU 上临时放东西的工作台。训练时显存里不只放模型参数，还会放这一批样本的 token、每一层中间结果、attention 计算的临时矩阵、loss 反向传播需要的计算图、每个参数的梯度、优化器状态等。`batch_size` 越大、序列越长、层数越多，这张工作台上同时要摆的东西越多。工作台摆不下，就会出现 CUDA out of memory，也就是常说的 OOM。梯度累积的思路不是让工作台变大，而是把一次大活拆成几次小活：每次只摆一小批，摆得下；连续做几小批后，再汇总梯度更新参数。

可直接口述回答 (>=1000字)

MiniMind 预训练默认设置梯度累积，最直接的原因是：它想在单次显存只能装下较小 micro-batch 的情况下，模拟一个更大的有效 batch。预训练入口里默认 `batch_size=32`，`accumulation_steps=8`，位置分别在 [train_pretrain.py](#) 和 [train_pretrain.py](#)。这里的 `batch_size` 不是最终每次参数更新看到的全部样本数，而是每次塞进模型的一小批样本数。训练循环在 [train_pretrain.py](#) 到 [train_pretrain.py](#) 做的是：先把这一批 token 喂给模型，让模型猜下一个 token，这叫 forward；再把猜测和答案对比，得到 loss；接着把 loss 除以 `args.accumulation_steps`；最后执行 backward，把“这批数据建议参数怎么改”累积到 `.grad` 里。只有当 `step % args.accumulation_steps == 0` 时，才会在 [train_pretrain.py](#) 到 [train_pretrain.py](#) 做梯度裁剪、`optimizer.step()` 和 `optimizer.zero_grad()`。也就是说，backward 像写修改建议，`optimizer.step()` 才是真的修改模型参数，`optimizer.zero_grad()` 是把已经用过的建议清掉，避免下一轮把旧建议也算进去。

可以把这个过程想成“分 8 次搬砖，再一起砌墙”。如果显存是一辆小推车，一次只能装 32 条样本的激活、梯度和临时张量，那就不要硬把 256 条样本一次塞进去。MiniMind 的做法是：每次推 32 条样本进模型，得到一份梯度，不马上更新参数，而是先把梯度存在参数的 `.grad` 里；连续推 8 次以后，再让 AdamW 根据累积起来的梯度更新一次模型参数。单进程下，有效 batch 近似是：

$$B_{\text{effective}} = B_{\text{micro}} \times K$$

其中 B_{micro} 是 `batch_size`， K 是 `accumulation_steps`。按默认值就是：

$$B_{\text{effective}} = 32 \times 8 = 256$$

如果考虑序列长度，上游默认 `max_seq_len=340`，那么单进程一次 optimizer update 大约覆盖：

$$T_{\text{effective}} = B_{\text{micro}} \times K \times L = 32 \times 8 \times 340 = 87040$$

个 token 位置。这里 L 是序列长度。若启用 DDP，也就是 `DistributedDataParallel`，中文可以理解成“分布式数据并行训练”，有效 batch 还要乘以 `world_size`：

$$B_{\text{effective}} = B_{\text{micro}} \times K \times W$$

其中 W 是 GPU 训练进程数。比如有 2 张 GPU，每张 GPU 各有一个训练进程，每个进程都处理自己的 batch，反向传播后再同步梯度，那么 $W=2$ 。本机当前主要关注单机小显存学习场景，不要把 DDP 写成已经验证过的多卡训练成果。

为什么 loss 要除以 `accumulation_steps`？这里要抓住一个 PyTorch 习惯：`loss.backward()` 默认不是“覆盖旧梯度”，而是“把新梯度加到旧梯度上”。这就是为什么普通训练每次 `optimizer.step()` 之后都要 `optimizer.zero_grad()`。如果不清空，下一批数据的梯度会继续加在上一批的 `.grad` 上，优化器以为这些梯度都属于当前要更新的一次决策。梯度累积正是故意利用这个“会相加”的机制：前 7 次不清空，让梯度攒起来；第 8 次更

新完才清空。问题在于，既然要把 8 批数据当作一个大 batch，我们通常想取“8 批的平均意见”，不是“8 批意见直接叠成 8 倍音量”。所以每一批的 loss 先除以 8，再 backward。源码里 [train_pretrain.py](#) 先取 `res.loss + res.aux_loss`，再在 [train_pretrain.py](#) 除以累积步数。公式可以写成：

$$L_i = L_{\text{CE},i} + L_{\text{aux},i}$$

$$L_{\text{backward},i} = \frac{L_i}{K}$$

连续 K 个 micro-batch 的累积梯度是：

$$g = \sum_{i=1}^K \nabla_{\theta} \left(\frac{L_i}{K} \right) = \frac{1}{K} \sum_{i=1}^K \nabla_{\theta} L_i$$

这就近似等价于对 K 个 micro-batch 的平均 loss 做一次反向传播。直观说，8 份作业分别批改，最后取平均意见再改模型，而不是让第 8 份意见独断专行，也不是把 8 份意见暴力叠成 8 倍音量。

`SkipBatchSampler` 解决的是另一个问题：断点续训时，模型、优化器和 scaler 状态恢复了，数据进度也要恢复。它的实现位于 [trainer_utils.py](#)。训练入口在每个 epoch 内先准备样本索引：DDP 模式用 `DistributedSampler`，普通单进程用 `torch.randperm(len(train_ds)).tolist()`，对应 [train_pretrain.py](#) 和 [train_pretrain.py](#)。然后它计算：

```
skip = start_step if (epoch == start_epoch and start_step > 0) else 0
batch_sampler = SkipBatchSampler(train_sampler or indices, args.batch_size, skip)
```

这两行在 [train_pretrain.py](#) 和 [train_pretrain.py](#)。如果 checkpoint 里记录 `step=200`，恢复训练时就让 sampler 丢掉本 epoch 前 200 个已经训练过的 batch，从第 201 个 batch 继续交给 DataLoader。它跳过的是 batch，不是 token，也不是单条样本，更不是 optimizer update 次数。

比喻一下，训练像读一本很厚的题册。checkpoint 里保存了你当前写到第几页、笔的墨水状态、老师批改风格，也就是模型参数、optimizer 状态、scaler 状态、epoch 和 step；`SkipBatchSampler` 像书签，它告诉 DataLoader：“前面这些页上次已经做过了，别又从第一页开始。”如果没有它，恢复训练虽然加载了模型参数，但数据会从 epoch 开头再喂一遍，等于同一批样本被重复训练；如果跳过太多，又会漏掉还没训练过的样本。两者都会让训练轨迹和原本连续训练不一致。

它的核心逻辑很简单：[trainer_utils.py](#) 逐个遍历样本下标，把下标塞进 `batch`；[trainer_utils.py](#) 凑够 `batch_size` 后，如果当前跳过数量还小于 `skip_batches`，就丢弃这个 batch；否则 `yield batch` 交给 DataLoader。最后如果还有不满一个 batch 的尾巴，并且已经完成跳过，也会在 [trainer_utils.py](#) 到 [trainer_utils.py](#) 交出去。

把它和梯度累积放在一起看，关系是这样的：`SkipBatchSampler` 决定“这次训练从哪些样本下标组成的 micro-batch 开始”；梯度累积决定“几个 micro-batch 后才真正更新一次参数”。前者管数据进度，后者管更新节奏。它们共同服务预训练主链路：

```
JSONL text
-> tokenizer
-> PretrainDataset 生成 input_ids / labels
-> SkipBatchSampler 组织和跳过 batch 下标
-> DataLoader 取出 micro-batch
-> MiniMindForCausalLM.forward 计算 logits 和 loss
-> loss / accumulation_steps
-> backward 累积梯度
-> 每 K 个 step 做 optimizer.step
-> 保存普通权重和 resume checkpoint
```

在 RTX 5060 Laptop 约 8GB 显存上，这两个设计尤其现实。8GB 显存不像大显存训练卡那样可以随便提高 batch 和序列长度。梯度累积让我们先用更小的 `batch_size` 和 `max_seq_len` 跑通，再通过 `accumulation_steps` 调整有效 batch；`SkipBatchSampler` 则让长时间训练被中断后不至于从头浪费计算。但截至当前文档，本仓库只验证过 CPU 极小单 batch 闭环，没有验证正式默认配置在这张 GPU 上能稳定跑。因此能保守写进 README 或面试材料的是：“我阅读并同步了 MiniMind 预训练中梯度累积和断点跳 batch 的源码，理解它们分别解决显存和续训进度问题，并完成过极小训练闭环验证。”不能写成“已经完成默认配置预训练”或“验证了默认梯度累积在 8GB GPU 上稳定收敛”。

详细原理讲解（>=3000字，含公式）

1. 先从预训练到底在做什么说起

MiniMind 的预训练，本质上是在做因果语言模型训练。给模型一串 token，让它在每个位置预测下一个 token。比如一句话被 tokenizer 变成：

```
[BOS, 今, 天, 天, 气, 好, EOS, PAD, PAD, ...]
```

`PretrainDataset` 的行为位于 [lm_dataset.py](#)。它读取 JSONL 的 `text` 字段，在 [lm_dataset.py](#) 用 tokenizer 编码文本，在 [lm_dataset.py](#) 手工加 BOS 和 EOS，在 [lm_dataset.py](#) padding 到固定长度。然后它复制一份 `input_ids` 作为 `labels`，并在 [lm_dataset.py](#) 把 PAD 位置的 label 改成 `-100`。这一步的意思不是“PAD 从输入里消失了”，而是“PAD 对应的位置不参与 cross entropy loss”。

模型 forward 的 loss 计算在 [model_minimind.py](#)。当 `labels` 不为空时，[model_minimind.py](#) 做了一次 shift：

```
x, y = logits[..., :-1, :].contiguous(), labels[..., 1:].contiguous()
```

也就是用第 t 个位置的输出预测第 $t+1$ 个 token。公式写成：

$$x_{b,t,:} = \text{logits}_{b,t,:}$$

$$y_{b,t} = \text{labels}_{b,t+1}$$

其中 b 表示 batch 中第几个样本， t 表示序列位置，冒号 `:` 表示词表维度上的全部 logits。`logits` 的 shape 是：

```
[batch_size, seq_len, vocab_size]
```

`labels` 的 shape 是：

```
[batch_size, seq_len]
```

shift 后:

```
x.shape = [batch_size, seq_len - 1, vocab_size]
y.shape = [batch_size, seq_len - 1]
```

交叉熵可以写成:

$$L_{\text{CE}} = -\frac{1}{N} \sum_{(b,t) \in \Omega} \log \frac{\exp(x_{b,t,y_{b,t}})}{\sum_{v=1}^V \exp(x_{b,t,v})}$$

这里 V 是词表大小, Ω 是所有没有被 `ignore_index=-100` 忽略的位置集合, $N=|\Omega|$ 是有效预测位置数量。直观理解: 模型在每个位置给词表里每个 token 打分, 正确答案那个 token 的概率越高, loss 越低; PAD 位置被 `-100` 排除, 不进入平均。

从训练循环看, MiniMind 的链路是:

```
input_ids, labels
-> model(input_ids, labels=labels)
-> res.loss + res.aux_loss
-> loss / accumulation_steps
-> backward
-> 若到累积边界则 clip + optimizer.step + zero_grad
```

这条链路的关键不是“跑了多少个 for 循环”, 而是“什么时候只累积梯度, 什么时候真正改参数”。

2. 把 forward、loss、backward、step、zero_grad 拆开讲

先看一句 MiniMind 训练代码:

```
res = model(input_ids, labels=labels)
```

这句就是 forward。它不是一个神秘动作, 只是调用模型的 `forward` 函数。 `input_ids` 是一批 token id, shape 大概是:

```
[batch_size, seq_len]
```

如果 `batch_size=32`、`seq_len=340`, 那就是 32 行、每行 340 个 token id。模型拿到这些数字后, 会经过 embedding、Transformer block、RMSNorm、`lm_head`, 最后输出 `logits`。 `logits` 可以理解成“模型对每个位置的下一个 token 候选打了多少分”。shape 是:

```
[batch_size, seq_len, vocab_size]
```

如果词表大小是 6400, 那么每个位置都有 6400 个分数。比如第 10 个位置, 模型不是只输出一个答案, 而是给词表里 6400 个 token 都打分: 这个像正确答案, 那个不像正确答案。

接下来是 loss。MiniMind 的 `MiniMindForCausalLM.forward` 在 [model_minimind.py](#) 做 shifted cross entropy。简单说，模型在位置 0 的输出要预测位置 1 的 token，位置 1 的输出要预测位置 2 的 token。loss 就是把“模型给正确 token 的分数”变成一个错误程度。正确 token 概率越高，loss 越低；正确 token 概率越低，loss 越高。

再看：

```
scaler.scale(loss).backward()
```

为了先理解主线，可以暂时把 `scaler.scale(...)` 当成混合精度训练里的包装，核心动作是 `loss.backward()`。它做的事情是：从 loss 这个错误数字出发，沿着刚才 forward 建出来的计算图往回走，算出每个参数对 loss 的影响。算出来的结果不会直接改参数，而是写入每个参数自己的 `.grad` 字段。

一个极简类比：

```
参数 theta = 当前菜谱
loss = 顾客觉得难吃的程度
backward = 分析这道菜到底是盐多了、火大了，还是糖少了
.grad = 写在纸上的修改建议
optimizer.step = 厨师真的改菜谱
zero_grad = 把这次建议擦掉，准备记录下一轮顾客反馈
```

这就解释了为什么 `loss.backward()` 和 `optimizer.step()` 是两回事。`backward` 只是算建议，`step` 才是执行建议。中间可以插入很多工程动作，比如梯度裁剪、梯度累积、混合精度反缩放、多卡梯度同步。

最后看：

```
optimizer.zero_grad(set_to_none=True)
```

它为什么必须有？因为 PyTorch 的设计是：每次 backward 得到的新梯度，会加到参数已有的 `.grad` 上，而不是自动覆盖。这个设计不是错误，它让梯度累积成为可能。但普通训练里，如果每一批数据都应该独立更新一次，就必须在更新后清空旧梯度。否则下一批数据 backward 时，`.grad` 里会混着上一批的旧建议。

用一个只有一个参数的玩具例子看：

```
第 1 批数据 backward 后: theta.grad = 0.3
optimizer.step 根据 0.3 更新参数
如果不 zero_grad
第 2 批数据 backward 算出新梯度 0.2
theta.grad 会变成 0.3 + 0.2 = 0.5
optimizer.step 会按 0.5 更新
```

这样第 2 次更新就不再只代表第 2 批数据，而是混进了第 1 批已经用过的梯度。普通训练中这通常是 bug。梯度累积中，这个“相加”是故意利用的，但必须有边界：攒够 `accumulation_steps` 后更新一次，然后清空，开始下一组累积。如果不清空，第 1 组的 8 批梯度还会混进第 2 组，第 2 组又混进第 3 组，训练就会变成越滚越大的旧账本。

3. 什么是梯度，为什么 backward 不等于参数更新

模型参数可以抽象成一个很长的向量：

$$\theta = [\theta_1, \theta_2, \dots, \theta_m]$$

loss 是参数的函数：

$$L = f(\theta)$$

反向传播计算的是梯度：

$$g = \nabla_{\theta} L = \left[\frac{\partial L}{\partial \theta_1}, \frac{\partial L}{\partial \theta_2}, \dots, \frac{\partial L}{\partial \theta_m} \right]$$

梯度的直观意义是：如果某个参数朝某个方向变化，loss 会怎么变。你可以把梯度想成山坡上的坡度箭头。模型训练不是一次就飞到山脚，而是一边摸坡度，一边小步下山。

PyTorch 里 `loss.backward()` 的职责是把梯度算出来，并累加到每个参数的 `.grad` 上。它不直接更新参数。真正改变参数的是 optimizer，例如 AdamW 的 `optimizer.step()`。MiniMind 预训练里，backward 在 [train_pretrain.py](#)，参数更新在 [train_pretrain.py](#)，清空梯度在 [train_pretrain.py](#)。所以一个高频误解要先纠正：

backward：把“这批数据建议参数怎么改”的意见写到 `.grad`
optimizer.step：真正按这些意见修改参数
zero_grad：清空意见箱，准备下一轮

如果每次 backward 后立刻 step，这就是普通的小 batch 训练。MiniMind 默认没有这样做，而是让多个 micro-batch 先把意见写进同一个 `.grad`，等到第 8 次再统一更新。

4. 显存到底在训练里起什么作用

显存可以理解成 GPU 的高速工作台。CPU 内存也能放数据，但 GPU 做矩阵乘法很快，所以训练时要把模型和当前 batch 搬到 GPU 显存里。显存不够时，GPU 不是慢一点，而是很多情况下直接报 OOM，因为中间结果没有地方放。

训练时显存里常见几类东西：

1. 模型参数：每一层权重，比如 embedding、attention、MLP 的矩阵。
2. 当前 batch 的输入：input_ids、labels，以及后续变成的 embedding。
3. forward 中间激活：每一层算出来的 hidden states，backward 要用它们反推梯度。
4. attention 临时张量：例如注意力分数矩阵，序列越长越贵。
5. 梯度：每个可训练参数通常都有一份对应的 grad。
6. 优化器状态：AdamW 会为参数维护动量和二阶矩，训练时也占空间。
7. 混合精度或框架临时缓存：包括 CUDA kernel 工作区、缓存分配等。

为什么 batch_size 和 seq_len 对显存影响大？因为它们决定“这次同时放上工作台的数据量”。batch_size 从 4 变成 8，很多激活大致也会跟着翻倍。seq_len 从 256 变成 512，不只是 token 数翻倍，attention 里的某些中间计算还可能接近按序列长度平方增长。MiniMind 默认 max_seq_len=340，对小模型还算克制，但在 8GB 笔记本 GPU 上仍然不能直接默认安全。

梯度累积解决的是这个问题：如果你想让一次参数更新综合 256 条样本的意见，但显存一次只能放 32 条，就分 8 次放。每次 forward/backward 都只需要承受 32 条样本的显存压力；8 次之后，`.grad` 里已经攒了 8 批样本的平均梯度，再更新参数。这就是“省单次显存，不省总计算量”。它像小桌子包饺子：桌子一次只能摆 32 个饺子皮，那就摆 8 轮，最后一起下锅；桌子没有变大，工作次数变多了。

注意它不解决所有显存问题。模型参数本身、optimizer 状态、单条样本序列太长、某一层 attention 太大，这些仍然会占显存。如果 `batch_size=1` 都 OOM，单纯增加 `accumulation_steps` 没用，因为单次最小 micro-batch 也放不下。那就要减 `max_seq_len`、减模型规模、换 dtype、做激活检查点或换更大显存。

5. 为什么需要梯度累积

LLM 训练很吃显存。一次 forward/backward 不只存模型参数，还要存中间激活、attention 临时张量、梯度、optimizer 状态等。粗略讲，影响显存的关键变量包括：

```
模型大小
batch_size
seq_len
hidden_size
num_hidden_layers
attention heads
dtype
是否保存 optimizer state
是否使用 activation checkpointing
```

MiniMind 默认 `hidden_size=768`、`num_hidden_layers=8`、`batch_size=32`、`max_seq_len=340`，这些默认参数在 [train_pretrain.py](#) 到 [train_pretrain.py](#)。对于 RTX 5060 Laptop 约 8GB 显存来说，直接照搬完整配置是有风险的。已有实验记录只证明极小 CPU 单 batch 能跑通，见 [实验记录](#)，不能证明默认 GPU 训练一定可承受。

梯度累积的思路是：一次装不下大 batch，就把大 batch 切成几小份。每份叫 micro-batch。每个 micro-batch 都正常 forward/backward，但暂时不 step。等凑够 K 份后再 step。

设第 i 个 micro-batch 的 loss 是 L_i ，累积步数是 K 。如果我们想模拟“把 K 个 micro-batch 合成一个大 batch 后取平均 loss”，目标 loss 是：

$$L_{\text{big}} = \frac{1}{K} \sum_{i=1}^K L_i$$

它对参数的梯度是：

$$\nabla_{\theta} L_{\text{big}} = \nabla_{\theta} \left(\frac{1}{K} \sum_{i=1}^K L_i \right) = \frac{1}{K} \sum_{i=1}^K \nabla_{\theta} L_i$$

MiniMind 在每个 micro-batch 里先做：

$$L_{\text{backward},i} = \frac{L_i}{K}$$

然后 backward。由于梯度会累加，所以 K 次 backward 后 `.grad` 里得到的就是：

$$g_{\text{accumulated}} = \sum_{i=1}^K \nabla_{\theta} \left(\frac{L_i}{K} \right) = \frac{1}{K} \sum_{i=1}^K \nabla_{\theta} L_i$$

这正好对应平均 loss 的梯度。源码对应 [train_pretrain.py](#)。

如果不除以 K ，累积梯度会变成：

$$g_{\text{wrong}} = \sum_{i=1}^K \nabla_{\theta} L_i = K \cdot g_{\text{average}}$$

这会让实际更新幅度约等于放大 K 倍。它不一定马上报错，但就像把方向盘灵敏度突然调成 8 倍，训练很可能变得更难稳定。尤其在 LLM 预训练里，loss、梯度、学习率、warmup/cosine 调度、AdamW 动量都耦合在一起，偷放大数据率会让问题更隐蔽。

一个更生活化的比喻：假设 8 个同学给同一篇作文提修改意见。正确的做法是把 8 份意见平均一下，得到一个综合建议；错误做法是把 8 份意见不加平均直接叠在一起，结果同一句话被批 8 次，作者可能把整段都删掉。`loss / accumulation_steps` 就是在做“平均意见”。

6. MiniMind 里梯度累积的具体节奏

训练循环从 [train_pretrain.py](#) 开始。核心几行是：

```
res = model(input_ids, labels=labels)
loss = res.loss + res.aux_loss
loss = loss / args.accumulation_steps
scaler.scale(loss).backward()

if step % args.accumulation_steps == 0:
    scaler.unscale_(optimizer)
    torch.nn.utils.clip_grad_norm_(model.parameters(), args.grad_clip)
    scaler.step(optimizer)
    scaler.update()
    optimizer.zero_grad(set_to_none=True)
```

`res.loss` 是语言模型交叉熵，`res.aux_loss` 是 MoE 辅助 loss。当前默认 `use_moe=0`，所以 dense 模型下辅助 loss 是 0 或等价零张量；如果启用 MoE，它会参与总 loss。总 loss 公式是：

$$L_i = L_{\text{CE},i} + L_{\text{aux},i}$$

backward 用的是：

$$L_{\text{backward},i} = \frac{L_{\text{CE},i} + L_{\text{aux},i}}{K}$$

然后每第 K 个 step 做一次更新。默认 $K=8$ ，所以 step 1 到 7 只 backward 不 step；step 8 先把前 8 个 micro-batch 的梯度合在一起，再做 `optimizer.step()`；step 9 到 15 又只累积；step 16 再更新。

表格化理解：

```
step 1: forward/backward, 存梯度, 不更新
step 2: forward/backward, 继续累积, 不更新
...
step 8: forward/backward, 裁剪梯度, optimizer.step, zero_grad
step 9: 新一轮累积开始
```

训练日志里的 `step` 是 micro-batch 计数，不是 optimizer update 计数。这个点很重要。如果日志显示 step 800，默认累积步数为 8，那么理论上大约发生了 100 次 optimizer update。保存间隔 `save_interval=1000` 也按 micro-batch step 判断，见 [train_pretrain.py](#)。因此复盘训练进度时，不要把 `step` 直接等同于参数更新次数。

源码还有一个尾部处理：[train_pretrain.py](#) 到 [train_pretrain.py](#)。如果一个 epoch 结束时，最后剩余的 micro-batch 数量不足 `accumulation_steps`，代码仍然会做一次 `optimizer.step()`，避免最后几批数据的梯度完全白算。

尾部窗口为什么仍然除以 K，而不是改除以 r

先把两个数字分开：

- K ：完整梯度累积窗口的长度，例如默认 `accumulation_steps=8`。
- r ：当前 epoch 最后实际剩下的 micro-batch 数量，且 $0 < r < K$ 。

当前源码的明确事实是：每一个 micro-batch 都在进入 `backward()` 前执行同一行 `loss = loss / args.accumulation_steps`。它没有在 epoch 最后专门把除数改成 r ，也没有在最后一步对已累计的梯度重新乘一个系数。因此，尾部进入优化器的原始累计梯度是：

$$G_{tail} = \frac{1}{K} \sum_{i=1}^r \nabla_{\theta} L_i$$

如果希望把最后这 r 批也当作一个独立的、平均化的小窗口，才会使用：

$$G_{tail,avg} = \frac{1}{r} \sum_{i=1}^r \nabla_{\theta} L_i$$

上游源码没有写注释解释为什么选前一种；下面只能作为工程推断，而不是源码作者明示的设计理由：统一始终除以 K 的实现最简单，训练循环不需要在进入最后一组之前先统计剩余批数，也不需要已经完成多次 `backward()` 之后再把所有参数的 `.grad` 统一乘以 K/r 。它保证了每个 micro-batch 在整轮训练中都按相同权重写入梯度缓冲区。

用默认 $K=8$ 、尾部 $r=3$ 举例。假设三个原始梯度分别是 2 、 4 、 6 ：

$$G_{tail} = \frac{2 + 4 + 6}{8} = 1.5$$

而按尾部自己的平均值计算会是：

$$G_{tail,avg} = \frac{2 + 4 + 6}{3} = 4$$

因此，二者的原始梯度关系为：

$$G_{tail} = \frac{r}{K} G_{tail,avg}$$

这里是 $3/8$ 。这回答了“影响是什么”：**尾部窗口送入优化器的原始累计梯度，按数值上会比按 r 平均的版本小 r/K 倍。**

不过要避免一句过度简化的话：不能直接断言 MiniMind 最终的参数移动距离也必然严格缩小 r/K 。如果优化器是没有动量、没有梯度裁剪的普通 SGD，这个结论成立；但 MiniMind 使用 AdamW，还会在更新前做梯度裁剪。AdamW 会利用一阶与二阶动量统计，梯度缩放对参数实际移动的影响不是一个始终固定的比例。准确说法是：**尾部 raw gradient 的归一化方式不同，会影响 AdamW 接收到的梯度、动量状态和可能发生的裁剪；具体参数差异需要实验测量。**

为什么这通常能接受？在很长的训练中，尾部最多只占一个不完整窗口，影响会被大量完整窗口稀释；但在极小数据集、每个 epoch 只有十几个 micro-batch 的实验里，这个尾部占比会明显变大。当前极小 CPU 单 batch 实验没有覆盖正式训练循环和该尾部路径，所以这一段属于“源码事实 + 工程判断”，不是本机训练验证结果。

若未来要让尾部按 r 平均，常见思路只有两类：提前知道最后窗口大小并让这 r 个 loss 都除以 r ，或在最后更新前把已累计梯度统一乘以 K/r 。但会影响 AMP、梯度裁剪、resume 边界和测试口径，不能为了公式更漂亮就直接改，应该先写最小对照实验。

7. 再用数字讲一遍为什么 loss 要除以 accumulation_steps

前面的公式可能还是抽象，这里用一个只有一个参数的例子。假设模型只有一个参数 θ ，学习率是 0.1 ，我们攒 4 个 micro-batch 再更新，先不考虑 AdamW 的动量细节，把优化器简化成普通梯度下降。

4 个 micro-batch 算出来的原始梯度分别是：

```
g1 = 2
g2 = 4
g3 = 6
g4 = 8
```

如果我们真的把这 4 批合成一个大 batch，并对 loss 取平均，那么平均梯度应该是：

$$g_{\text{avg}} = \frac{2 + 4 + 6 + 8}{4} = 5$$

参数更新量大约是：

$$\Delta\theta = -0.1 \times 5 = -0.5$$

MiniMind 用梯度累积模拟这个效果。每批 loss 先除以 4，因此每批贡献到 `.grad` 的梯度也相当于除以 4：

```
第 1 批贡献: 2 / 4 = 0.5
第 2 批贡献: 4 / 4 = 1.0
第 3 批贡献: 6 / 4 = 1.5
第 4 批贡献: 8 / 4 = 2.0
累积起来: 0.5 + 1.0 + 1.5 + 2.0 = 5.0
```

这和大 batch 平均梯度一致。

如果不除以 4：

```
累积梯度 = 2 + 4 + 6 + 8 = 20
参数更新量 = -0.1 * 20 = -2.0
```

同样的学习率，更新幅度从 `-0.5` 变成 `-2.0`，刚好放大 4 倍。这就是“偷偷放大学习率”的意思。它不是说代码里的学习率数字真的变了，而是优化器拿到的梯度大了，最终参数移动距离变大了。学习率和梯度在参数更新里通常是相乘关系：

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \cdot g$$

其中 η 是学习率， g 是梯度。 g 放大 4 倍，效果就像 η 放大 4 倍。AdamW 比这个公式复杂，但“梯度尺度变大，会改变更新尺度和优化器状态”这个直觉仍然成立。

所以 `loss / accumulation_steps` 不是为了让日志上的 loss 好看，也不是为了让模型少学习，而是为了保持“大 batch 平均 loss”的语义。MiniMind 的日志在 [train_pretrain.py](#) 又乘回 `args.accumulation_steps`，是为了打印时看到原始 loss 尺度，不让读日志的人误会 loss 真的变小了。

8. 梯度累积和显存的关系，不要理解反了

梯度累积不等于“显存需求变成原来的 1/8”。它减少的是单次 forward/backward 需要同时放进显存的样本数。比如你想要有效 batch 256，但 8GB 显存装不下 `batch_size=256`，可以设置 `batch_size=32`，`accumulation_steps=8`。这样每次只处理 32 条，显存压力接近 micro-batch 32 的水平，而不是 256 的水平。

但是梯度累积也不是免费午餐。原先一句“墙钟时间会增加”太像机翻，下面把它拆开。

墙钟时间到底是什么

“墙钟时间”就是你从按下回车开始，到终端打印训练结束为止，现实世界里钟表走过的时间，也可叫“实际耗时”或“经过时间”。它不只包括 GPU 真正在算矩阵乘法的时间，还包括数据加载、CPU 等待、GPU 同步、保存 checkpoint、日志输出等。

要先说明比较对象，否则“会不会更慢”没有唯一答案：

- 固定一次 `optimizer.step()` 来看：`accumulation_steps=8` 意味着要连续完成 8 次 forward/backward 才更新一次参数，所以这一次更新前经过的实际时间通常比只跑 1 次 forward/backward 更长。
- 固定总样本数或总 token 数来看：不做累积和做累积都要让每个样本各自经历一次 forward/backward，核心计算量并不会凭空多出 8 倍。梯度累积常常会因为 micro-batch 更小、GPU 利用率较低、Python 循环和数据加载次数更多而更慢，但到底慢多少必须测量，不能写成固定倍数。
- 与“能一次装下的大 batch”相比：梯度累积的价值恰恰是后者可能在 8GB 显存上根本无法运行。此时它不是在同一硬件上追求最快，而是在可训练与 OOM 之间选择可训练。

BatchNorm 为什么会让大 batch 与梯度累积不同

BatchNorm 会在每次 forward 时，根据“当前这一批输入”计算均值和方差。假设本来想一次把 256 条样本送进 BatchNorm，它会得到一套基于 256 条样本的统计量；而使用 8 个、每个 32 条的 micro-batch 时，它会分别得到 8 套不同的统计量。即使最后梯度相加方式正确，前向计算本身已经不同了，所以不能等价。

MiniMind 主体使用的是 RMSNorm，不是 BatchNorm；当前上游模型代码中可以看到 RMSNorm 模块和对应层的使用。因此上述 BatchNorm 差异不是 MiniMind 默认 Transformer 主线的主要风险。这个结论是源码结构阅读，不是本机训练性能实验。

Dropout 为什么会让轨迹不完全一样

Dropout 在训练时会随机把一部分激活置零。一次性大 batch 和分成 8 个 micro-batch，会经历不同次数、不同位置的随机掩码抽样；因此即使样本集合相同，也不保证每个参数更新逐 bit 相同。

MiniMind 代码里确实定义了 embedding、attention 与 residual 相关的 Dropout 模块；但 `MiniMindConfig` 当前默认 `dropout=0.0`。因此按默认配置创建模型时，Dropout 实际不生效，这一项在默认 MiniMind 预训练中不是主要差异来源。只有后续显式把 `dropout` 设为非零时，才需要把它作为明显差异考虑。

学习率调度按 micro-step 推进是什么意思

先记住两个计数器：

```
micro-step: 处理了第几个小 batch。
update-step: 执行了第几次 optimizer.step()。
```

当 $K=8$ 时，micro-step 1 到 8 只发生 1 次 update-step。MiniMind 在每个 micro-batch 开始时都计算并写入学习率：

```
lr = get_lr(epoch * iters + step, args.epochs * iters, args.learning_rate)
for param_group in optimizer.param_groups:
    param_group['lr'] = lr
```

因此，学习率曲线的横轴是 micro-step。例如第 1 到第 7 个 micro-step 虽然没有真正更新参数，但学习率变量仍然被重新计算；第 8 个 micro-step 执行 `optimizer.step()` 时，真正生效的是第 8 个 micro-step 对应的学习率。

这自动等于错误。学习率调度必须先约定“一个时间单位”是什么：按处理过的 micro-batch、按 token 数，还是按真正更新次数。MiniMind 当前选择的是按 micro-batch 推进。它的后果是：如果你把总训练步数理解成 optimizer 更新次数，就会误读学习率衰减速度；本脚本里总共有大约 $\text{总 micro-step} / K$ 次参数更新，但学习率曲线在全部 micro-step 内已经走完。以后做对照实验时，必须同时记录 `accumulation_steps`、总 micro-step、总 optimizer update 次数和学习率曲线口径。

更严谨的总述是：MiniMind 的梯度累积在“连续 K 个 micro-batch 的平均梯度”这一层面模拟更大有效 batch；但前向中的 BatchNorm、随机性、学习率调度单位、尾部不完整窗口、混合精度和数值舍入，都会使它不保证与一次性大 batch 逐 bit 相同。

9. SkipBatchSampler 先别想复杂，它只是“批次书签”

`SkipBatchSampler` 位于 `trainer_utils.py`。它继承 PyTorch 的 `Sampler`，但它不读取 JSONL，不 tokenize，不创建 tensor，也不算 loss。它只产出一组组样本下标。DataLoader 拿到这些下标后，才调用 Dataset 的 `__getitem__` 取 `input_ids`, `labels`。

核心代码可以概括成：

```
batch = []
skipped = 0
for idx in self.sampler:
    batch.append(idx)
    if len(batch) == self.batch_size:
        if skipped < self.skip_batches:
            skipped += 1
            batch = []
            continue
        yield batch
        batch = []
    if len(batch) > 0 and skipped >= self.skip_batches:
        yield batch
```

它的输入有三个：

- `sampler`: 样本下标来源, 可以是 DDP 的 `DistributedSampler`, 也可以是普通 Python list。
- `batch_size`: 几个样本下标组成一个 batch。
- `skip_batches`: 从开头跳过几个完整 batch。

它的输出是一批一批下标, 比如:

```
sampler = [8, 3, 5, 1, 9, 0, 2]
batch_size = 2
skip_batches = 0
输出: [8,3], [5,1], [9,0], [2]
```

如果 `skip_batches=2`:

```
跳过: [8,3], [5,1]
输出: [9,0], [2]
```

这就是“批次书签”。恢复训练时, 它不是说“从样本 id 2 开始”, 而是说“按照本 epoch 的同一套顺序, 前 2 个 batch 已经训练过了, 从第 3 个 batch 开始”。

10. 它为什么和 resume 绑定, 而不是普通训练也必须跳

普通从头训练时, `start_step=0`, 所以 [train_pretrain.py](#) 得到 `skip=0`, `SkipBatchSampler` 不跳过任何 batch, 只是正常按 `batch_size` 分组。

当 `--from_resume 1` 时, 训练入口会在 [train_pretrain.py](#) 调用 `lm_checkpoint(...)` 尝试读取 resume checkpoint。 `lm_checkpoint` 的加载模式位于 [trainer_utils.py](#), 如果找到 resume 文件, 就返回其中保存的状态。恢复部分在 [train_pretrain.py](#) 到 [train_pretrain.py](#): 加载 `model`、`optimizer`、`scaler`, 并恢复 `epoch` 和 `step`。

保存 resume 的位置在 [train_pretrain.py](#) 和 [trainer_utils.py](#)。resume 里有:

```
model
optimizer
epoch
step
world_size
wandb_id
scaler
```

这几样拼在一起, 才像一个真正的“训练现场快照”。只保存模型参数, 相当于只知道学生现在会多少题; 保存 optimizer 状态, 才知道 AdamW 的动量、二阶矩等历史信息; 保存 scaler, 才知道 fp16 动态缩放状态; 保存 epoch/step, 才知道数据走到哪里。

如果恢复时只加载模型和 optimizer, 不跳过已训练 batch, 会发生什么? 假设上次在 epoch 0 的 step 200 保存了 checkpoint。重启后, 如果 DataLoader 又从该 epoch 的第 1 个 batch 开始, 那么前 200 个 batch 会再训练一遍。这不是“更认真复习”, 而是改变了训练数据分布和训练轨迹。预训练通常对数据顺序、学习率节奏和 optimizer 状态很敏感, 这种重复会让结果不再对应“连续训练到 step 201”。

如果跳过太多, 比如本来只训练到 step 200, 却跳过 300 个 batch, 就漏掉了 100 个没训练过的 batch。

`SkipBatchSampler` 的目标就是让恢复后的数据消费进度尽量对齐中断前的进度。

checkpoint 是为了什么：能恢复什么，不能自动修复什么

checkpoint 的主要目标不是“让任何错误都自动消失”，而是把某个已经完成的训练状态保存到磁盘。它主要应对机器重启、终端断开、进程被杀、网络或远程会话中断、手动停止，以及长训练不希望从零开始的情况。MiniMind 的 README 也把 checkpoint resume 描述为适合长时间训练或不稳定环境下避免训练进度丢失的机制。

当前 MiniMind resume 文件保存的是模型参数、optimizer 状态、epoch、step、world size、日志运行标识，以及在调用处传入的 scaler 状态。它可以把训练带回“最近一次成功写入 checkpoint 的状态”，但不等于能自动治好所有异常：

- OOM：resume 不会增加显存。若恢复后仍用同一模型、同一 `batch_size` 和同一序列长度，通常还会再次 OOM；需要先缩小配置或定位显存峰值。
- `loss=NaN`：若最近 checkpoint 本身仍然是有限数值状态，可以退回该 checkpoint 后排查学习率、数据、混合精度或梯度爆炸；若保存进去的模型或 optimizer 状态已经是 NaN，resume 只会把 NaN 原样读回来。
- checkpoint 之前尚未保存的工作：会丢失。恢复会从最近一次成功保存的位置继续，而不是从故障发生的精确 CPU/GPU 指令继续。

一个 micro-batch 尚未完整训练就中断，恢复时会跳过还是重跑

这里必须把“中断发生在什么位置”讲清楚。一个 micro-batch 不是一个可以随时保存的原子块；forward、backward、optimizer 更新、checkpoint 写盘是多个独立动作。当前 MiniMind 不保存参数 `.grad` 缓冲区，也不保存某个 micro-batch 的中间激活。因此它不能从“第 3 个 micro-batch 的 backward 算到一半”精确接着算。

中断位置	最近 checkpoint 中是否包含这批数据的效果	恢复后的实际倾向
还没完成 forward，或 forward/backward 途中直接崩溃	不包含	从最近 checkpoint 恢复，这个 batch 会重新训练，不会被跳过。
已完成 backward，但尚未到本轮 <code>optimizer.step()</code>	仅存在于进程内存的 <code>.grad</code> ，默认不会写入 resume 文件	若没有新 checkpoint 成功写入，恢复会回到旧 checkpoint，这个 batch 会重新训练；这通常比丢失它更安全。
已完成 <code>optimizer.step()</code> ，但新的 checkpoint 尚未写入	内存模型已经更新，但磁盘最近 checkpoint 仍旧	恢复会回到旧 checkpoint，之后重新做这部分工作。
checkpoint 已完整写入	包含到该保存点为止的模型、optimizer、step 等状态	<code>SkipBatchSampler</code> 会按保存的 step 跳过已经记录为完成的 batch。

所以对你的原问题，最简答案是：**不是“发现一个 batch 没训练完就直接跳过”，而是以最近一次真正保存成功的 checkpoint 为准。保存点之后、故障发生之前的工作一般会重跑；保存点已经记录为完成的 batch 才会被跳过。**

还有一个需要主动记录的源码边界。默认 `save_interval=1000`，默认 `$K=8$`，而 `$1000$` 能被 `$8$` 整除，因此常规间隔保存通常恰好发生在一次完整的 `optimizer.step()` 与 `zero_grad()` 之后，这是相对安全的对齐方式。可是脚本还会在 `step == iters` 时保存；当一个 epoch 的总 micro-step 不能被 `$K$` 整除，保存逻辑发生在循环内，而尾部补做的 `optimizer.step()` 在循环结束后。按当前代码顺序，终点 checkpoint 可能先记录 `step=iters`，随后才在内存中应用尾部梯度。若此时从这个 checkpoint 恢复，`SkipBatchSampler` 会跳过尾部 batch，但 checkpoint

中未必包含尾部更新。这里是从源码控制流得到的潜在续训语义风险，不是本机已经复现的 bug；应通过一个 `$iters \bmod K \neq 0$` 的最小 resume 对照实验确认，再决定是否调整保存时机或保存未完成的累计梯度。

11. SkipBatchSampler 和 DDP 的关系

DDP 是 `DistributedDataParallel` 的缩写，中文可以叫“分布式数据并行”。不要被名字吓到，它的核心想法很朴素：如果有多张 GPU，就让每张 GPU 都放一份相同的模型，但每张 GPU 看不同的数据。比如有 2 张 GPU、一个 batch 里总共想看 64 条样本，那么可以让 GPU 0 看其中 32 条，GPU 1 看另外 32 条。两边各自 forward、算 loss、backward。backward 后，每张 GPU 都得到自己那 32 条样本的梯度。然后 DDP 会把不同 GPU 的梯度做同步，通常是求平均或等价平均效果，让两份模型用一致的梯度更新。更新后，两张 GPU 上的模型参数仍然保持一致。

所以 DDP 的关键不是“模型被拆到多张 GPU 上”，而是“数据被分到多张 GPU 上，模型每张 GPU 一份”。这叫数据并行。与之相对，如果模型大到一张 GPU 放不下，把不同层拆到不同 GPU，那是模型并行，不是这里的 DDP 主线。

训练入口在 `train_pretrain.py` 判断是否初始化了分布式。如果有 DDP，就用 `DistributedSampler(train_ds)`；没有 DDP，就用普通随机索引列表。为什么需要 `DistributedSampler`？因为多张 GPU 不能每张都读完整数据。否则 GPU 0 训练样本 1 到 32，GPU 1 也训练样本 1 到 32，等于重复劳动。`DistributedSampler` 会按进程编号，也就是 rank，把数据索引分给不同进程。每个 epoch 开始时，`train_pretrain.py` 会调用 `train_sampler.set_epoch(epoch)`，这是 DDP 常见写法，用来让不同 epoch 的 shuffle 顺序发生变化。

单进程时，`train_pretrain.py` 用 `setup_seed(42 + epoch)` 后再 `torch.randperm(len(train_ds)).tolist()`。这意味着同一个 epoch、同一个 seed 下，随机索引顺序可以重现。恢复时 `start_epoch` 和 `start_step` 来自 checkpoint，只要代码、数据长度、seed 和采样逻辑不变，跳过前 `start_step` 个 batch 后，就能回到比较接近中断前的数据位置。

DDP 下还多一个 `world_size` 问题。`lm_checkpoint` 加载 resume 时会比较保存时的 `world_size` 和当前 `world_size`，如果不同，会在 `trainer_utils.py` 到 `trainer_utils.py` 转换 step：

$$\text{step}_{\text{new}} = \text{step}_{\text{old}} \times \frac{W_{\text{old}}}{W_{\text{new}}}$$

它的直觉是：GPU 数变多或变少后，每个进程负责的数据量变化了，所以本地 step 需要按进程数换算。这个换算是工程上的近似对齐，不应被描述成严格数学等价，尤其当数据不能整除、sampler padding、drop last 策略或 epoch 边界变化时，仍需要最小实验验证。

12. 两者放到同一张流程图里

`SkipBatchSampler` 和梯度累积经常同时出现，所以容易混。最清楚的办法是按时间顺序看：

1. 训练入口读取参数：batch_size=32, accumulation_steps=8
2. 读取或初始化模型：init_model
3. 读取 Dataset: PretrainDataset
4. 如果 from_resume=1, 读取 resume, 得到 start_epoch/start_step
5. 当前 epoch 内生成样本索引
6. SkipBatchSampler 根据 start_step 跳过已训练 batch
7. DataLoader 根据 sampler 输出的下标取出 micro-batch
8. 模型 forward, 得到 logits 和 loss
9. loss 除以 accumulation_steps
10. backward, 把梯度累到 .grad
11. 每 8 个 micro-batch 做一次 clip + optimizer.step + zero_grad
12. 到保存间隔时写普通权重和 resume checkpoint

换句话说：

`SkipBatchSampler`：决定从哪一批数据继续
梯度累积：决定攒几批梯度再更新
`checkpoint`：保存模型、优化器、`scaler`、`epoch`、`step` 等恢复所需状态

如果再用一个厨房比喻：`Dataset` 是食材仓库，`DataLoader` 是传菜员，`SkipBatchSampler` 是传菜单上的“前 200 桌已上过菜，从 201 桌开始”，梯度累积是厨师说“每 8 桌反馈汇总一次再改菜谱”。模型参数就是菜谱。
`optimizer.step()` 才是真正改菜谱的动作，前面的 `backward` 只是收集顾客反馈。

13. 高频易错点

第一，`batch_size` 不等于有效 batch。`MiniMind` 默认有效 batch 在单进程下近似为 `batch_size * accumulation_steps`，也就是 32 乘 8。若启用 DDP，还要乘以 `world_size`。

第二，`step` 不是 optimizer update 次数。源码中的 `step` 来自 `DataLoader` 的 `enumerate`，是 micro-batch step。默认每 8 个 step 才更新一次参数。

第三，`loss.item()` 日志做了还原。训练中 loss 被除以 `accumulation_steps`，日志在 [train_pretrain.py](#) 用 `loss.item() * args.accumulation_steps` 还原出当前 micro-batch 的原始尺度。不要看到 backward loss 被缩小，就误以为模型真实 loss 也被人为降低。

第四，`SkipBatchSampler` 跳的是 batch，不是样本，也不是 token。如果 `batch_size=32`，`skip_batches=10`，它会跳过前 10 个 batch，大约 320 条样本下标。最后一个尾 batch 可能不足 32 条。

第五，`ignore_index=-100` 只影响 loss，不会自动让 PAD 在 attention 里消失。当前预训练 `Dataset` 返回的是 `input_ids`，`labels`，训练调用只传 `model(input_ids, labels=labels)`，见 [train_pretrain.py](#)。这意味着 padding 位置不贡献 loss，但 PAD token 仍作为输入 token 进入模型。这个风险在 [源码指南](#) 也有记录，后续需要最小样本对照验证。

第六，断点续训不是只加载 `.pth` 权重。普通权重只保存模型参数，适合推理或阶段初始化；`resume checkpoint` 保存模型、optimizer、`scaler`、`epoch`、`step` 等，适合继续训练。二者目的不同。

第七，8GB 显存下不要直接拿上游默认训练配置当作已验证配置。合理做法是先极小 batch、短序列、少步数 smoke test，再逐步扩大；OOM 时记录参数组合和错误，不要把“源码默认”写成“本机可稳定训练”。

14. 最小验证该怎么做

当前已经有一份极小 CPU 单 batch 实验，证明最短训练闭环能跑通，但没有覆盖本专题两个重点。后续若要验证，应拆成两个小实验。

验证梯度累积：

固定 `seed`
构造 2 或 4 个 micro-batch
设置 `accumulation_steps=2`
记录 step 1 后参数不变但 `.grad` 存在
记录 step 2 后 `optimizer.step` 导致参数变化
对比 loss 除以 2 与不除以 2 的梯度范数差异

可验证的关键证据：

step 1: 参数最大变化 = 0, 梯度有限
step 2: 参数最大变化 > 0, zero_grad 后 .grad 被清空或置 None
梯度范数: 未缩放约为缩放版本的 2 倍

验证 SkipBatchSampler:

```
sampler = [8, 3, 5, 1, 9, 0, 2]
batch_size = 2
skip_batches = 2
期望输出 = [[9, 0], [2]]
```

如果进一步验证 resume, 需要跑若干 step 保存 checkpoint, 再重启并检查:

恢复的 epoch 与 step 正确
optimizer state 存在
scaler state 存在或按 dtype 合理为空/禁用
恢复后 DataLoader 第一批不是 epoch 开头第一批, 而是跳过 start_step 后的批次

这些验证如果未来完成, 可以写入新的实验记录; 当前不能提前写成已验证事实。

项目落地点

当前本地实现:

- 预训练入口: [train_pretrain.py](#)
- 梯度累积默认参数: [train_pretrain.py](#)
- loss 缩放与 backward: [train_pretrain.py](#)
- 每隔 accumulation_steps 更新参数: [train_pretrain.py](#)
- epoch 尾部不足累积窗口时的补 step: [train_pretrain.py](#)
- resume 加载入口: [train_pretrain.py](#)
- 恢复 model、optimizer、scaler、epoch、step: [train_pretrain.py](#)
- SkipBatchSampler 创建: [train_pretrain.py](#)
- SkipBatchSampler 实现: [trainer_utils.py](#)
- resume checkpoint 保存结构: [trainer_utils.py](#)
- 预训练 Dataset: [lm_dataset.py](#)
- shifted cross entropy: [model_minimind.py](#)

上游引用路径:

- 上游预训练入口: [train_pretrain.py](#)
- 上游训练工具: [trainer_utils.py](#)
- 上游 Dataset: [lm_dataset.py](#)
- 上游模型: [model_minimind.py](#)

已实现:

- 本地已存在与上游一致的预训练入口、Dataset、模型和训练工具源码。
- 本地已有极小 CPU 单 batch 训练闭环实验记录，验证过 forward、loss、backward、optimizer step 和最小 checkpoint/resume 文件结构。

正在设计或后续可扩展：

- 梯度累积最小实验：验证每个 micro-step 的参数变化、梯度存在性和 `loss / accumulation_steps` 对梯度范数的影响。
- `SkipBatchSampler` 单元测试：验证普通索引、尾 batch、`skip_batches` 超过总 batch 数等边界。
- resume 续训最小实验：保存后重启，确认恢复后第一批数据下标与预期一致。

需要验证：

- 正式 `trainer/train_pretrain.py` 在 RTX 5060 Laptop 约 8GB 显存上的可运行参数组合。
- CUDA + bf16/fp16 + GradScaler 分支。
- DDP 下 `world_size` 改变时 step 换算和 sampler 对齐的真实行为。
- 当 epoch 总 micro-step 不能被 `accumulation_steps` 整除时，尾部补 step 与 resume checkpoint 保存顺序的真实恢复行为。
- padding token 进入 attention 但不参与 loss 的影响边界。

可以写进 README、实验记录或面试材料的保守表述：

已阅读并同步 MiniMind 预训练入口，理解默认梯度累积通过 `loss` 缩放和延迟 `optimizer.step` 模拟更大有效 batch；理解 `SkipBatchSampler` 在 resume 时按 batch 跳过已消费数据，避免恢复训练重复喂数。当前已完成极小单 batch 训练闭环验证，正式梯度累积、DDP 和 GPU 长训练仍待专项验证。

暂时不能写的表述：

已经完成 MiniMind 默认配置预训练。
已经验证 `accumulation_steps=8` 在 RTX 5060 Laptop 8GB 上稳定收敛。
`SkipBatchSampler` 已在多卡 DDP resume 中验证完全等价连续训练。
梯度累积可以无代价替代真实大 batch。

面试官 / 评审者可能追问与回答

追问 1：为什么 MiniMind 要把 loss 除以 `accumulation_steps`，不除行不行？

不除也能运行，但训练语义变了。因为 PyTorch 的 backward 会把梯度累加到参数 `.grad` 上，连续 8 次 backward 如果不缩放 loss，累积梯度大约是平均梯度的 8 倍，相当于把有效学习率放大 8 倍。MiniMind 在 [train_pretrain.py](#) 做 `loss = loss / args.accumulation_steps`，让 K 个 micro-batch 的梯度累积后接近平均 loss 的梯度：

$$g = \frac{1}{K} \sum_{i=1}^K \nabla_{\theta} L_i$$

保守表达是：我已通过源码阅读确认了这个缩放逻辑，但还没有在正式训练入口里做专项梯度范数对照实验。

追问 2: `batch_size=32, accumulation_steps=8` 是不是等价于 `batch_size=256`?

在梯度平均这个核心意义上近似等价，但不是完全等价。有效 batch 可以按 $32 \times 8 = 256$ 理解；如果 DDP 还有 `world_size`，再乘以进程数。但 MiniMind 的学习率调度按 micro-batch step 推进，随机性、尾部不足累积窗口、混合精度和数值舍入都会导致轨迹不完全相同。所以更准确说法是：梯度累积用小显存模拟大有效 batch 的梯度规模，而不是保证和一次性大 batch 逐 bit 一致。

追问 3: `SkipBatchSampler` 跳过的是 step 还是 batch?

它跳过的是 batch。源码在 [trainer_utils.py](#) 凑够 `batch_size` 个样本下标后才判断是否跳过，所以 `skip_batches=2` 表示丢弃前两个完整 batch。训练入口把 resume 里的 `start_step` 传给它，见 [train_pretrain.py](#)。这里的 `step` 本身也是 DataLoader 产出的 micro-batch 计数，因此传进去后语义就是“跳过已经训练过的 micro-batch 数”。

追问 4: 只加载模型权重继续训练，和 `from_resume=1` 有什么区别?

只加载普通权重只恢复模型参数，不恢复 AdamW 的动量、二阶矩、GradScaler、epoch、step 和数据进度。`from_resume=1` 会通过 [trainer_utils.py](#) 保存/读取更完整的训练状态，并在 [train_pretrain.py](#) 到 [train_pretrain.py](#) 恢复这些状态。普通权重适合推理或从某个阶段初始化；resume 适合中断后继续训练。保守说法是：本仓库极小实验验证过最小 checkpoint/resume 文件结构，但正式入口的 resume + `SkipBatchSampler` 对齐仍需专项实验。

追问 5: 8GB 显存下应该怎么调这两个参数?

先不要照搬默认完整配置。现实顺序应该是：先降低 `batch_size`、`max_seq_len`、必要时降低模型层数或 hidden size，跑极小 GPU smoke test；如果单次 micro-batch 能放进显存，再用 `accumulation_steps` 提高有效 batch。比如从 `batch_size=1/2/4`、短序列和少步数开始，确认 forward、loss、backward、step 都正常，再逐步扩大。`SkipBatchSampler` 不直接省显存，它只服务 resume 数据进度。可以把它说成“断点续训的书签”，不能说成“显存优化手段”。

记忆点

梯度累积：小推车多跑几趟，再统一卸货更新参数。

loss / K: 把 K 趟货按平均重量结算，不让更新幅度偷偷放大。

`SkipBatchSampler`: 断点续训的书签，跳过已读过的 batch。

checkpoint: 不只是模型权重，还要保存 optimizer、scaler、epoch、step。

8GB 显存：先 smoke test，再扩大；源码默认不等于本机已验证。